

Anthony Zippo

Projects : ASL Detection Project

March

2026

Project Description

Brief Context:

-ASL is the primary language of the Deaf and Hard of Hearing community in the US . real world applications include communication aids, educational tools, and translation interfaces

My Program:

-A realtime American Sign Language hand sign recognition application using a webcam and a trained Convolutional Neural Network

-Detects hand signs frame by frame and displays the predicted ASL letter on screen with a confidence score

Project Description

- **Two-Part System:**

- **Training Component:** CNN trained offline in Google Colab on a large image dataset, saved as a model file

- **Inference Component:** live Python script running on local machine, captures webcam frames and feeds them to the model in real time

Project Description

Scope note:

- Covers the 26-letter ASL fingerspelling alphabet plus a space sign
- Full ASL vocabulary includes thousands of word level and motion based signs. Fingerspelling is a well defined learnable subset and a strong foundation

Architecture and Tools

Tools section:

- MediaPipe (Google) — detects hand in each webcam frame, identifies 21 landmark points across joints and fingertips, computes bounding box with padding. Handles the "find the hand" problem so the CNN focuses only on classification
- OpenCV — captures webcam feed frame by frame, passes frames to MediaPipe and model, draws bounding box and prediction text on screen
- TensorFlow / Keras — builds, compiles, trains, and loads the model. Keras provides a clean API for defining architecture and running training loops
- MobileNetV2 — lightweight CNN pre-trained on 1M+ ImageNet images. Used as frozen feature extractor via transfer learning. Backbone weights unchanged, new classification head trained specifically for ASL

Architecture and Tools

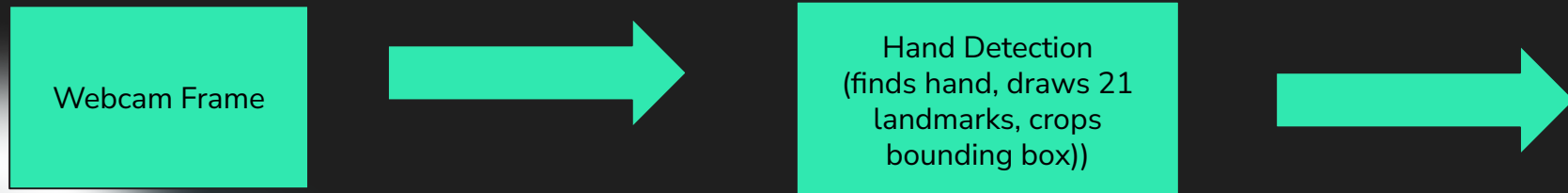
Transfer learning rationale:

- Training a CNN from scratch requires massive amounts of labeled data and significant compute time.
- To increase practicality of getting a working project we used MobileNetV2 since it already knows how to detect edges, textures, and shapes from ImageNet training
- **Freezing** the backbone and training only the head requires far less data and trains in minutes rather **than** hours or more

Architecture justified by AlexNet paper (Krizhevsky et al., 2012):

- Demonstrated CNN's learn hierarchical visual features: edges -> textures -> shapes -> object parts
- Finger positions and hand shapes are exactly the spatial patterns CNN's excel at recognizing

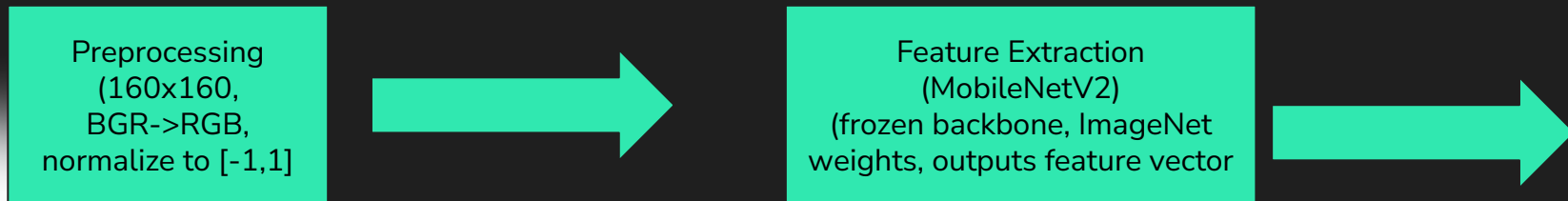
Pipeline



This is the raw input to the entire system. OpenCV opens your camera and reads it frame by frame in a continuous loop. Each frame is just a grid of pixels, typically 640x480, stored as numbers representing color values. One important detail: OpenCV stores color in BGR order (Blue, Green, Red) instead of the standard RGB order that most other software uses. This will matter in the future.

Before the CNN can classify anything, it needs to know where the hand is in the frame. MediaPipe solves this by identifying 21 landmark points across the hand. One for each fingertip, joint, knuckle, and wrist. From those 21 points it draws a bounding box around the hand with some padding, and that region gets cropped out of the full frame. This is important because without it the CNN would be looking at your entire room background, furniture, lighting, none of which is relevant to identifying a hand sign

Pipeline



Three things happen here in sequence, and all three must exactly match what was done during training or the model breaks:

First, the cropped hand region is resized to exactly 160×160 pixels because MobileNetV2 requires a fixed input size.

Second, the frame is converted from BGR to RGB to match how the training images were loaded.

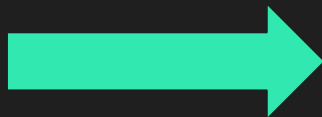
Third, pixel values are normalized from their raw range of 0–255 down to –1 to 1 using MobileNetV2's specific formula. This last step caused one of the biggest bugs in the project — using the wrong normalization at inference time caused the model to predict one letter at 100% confidence no matter what sign was shown, because it was receiving values it had never been trained on.

This is where the CNN does its work. MobileNetV2 was pre-trained on over 1 million images from ImageNet (a massive dataset covering thousands of categories) . Through that training it learned to detect visual patterns at every level of complexity: simple edges and color gradients in the early layers, textures and curves in the middle layers, and complex shapes and object parts in the deeper layers. These features transfer directly to ASL because recognizing hand shapes is fundamentally the same kind of spatial pattern detection problem.

The backbone weights are frozen (they do not change during our training). We are just borrowing the visual intelligence MobileNetV2 already learned and using it to describe our hand images. The output is a compact 1,280 number summary of what the network detected.

Pipeline

Classification Head
(Dense 512, Dense
256, Softmax 27
classes)



Predicted Letter+Confidence

(Smoothed over 5 frames, top
3 guesses shown on screen)

The 1,280 numbers from MobileNetV2 get fed into a small neural network that we trained specifically for ASL. It has two fully connected layers, 512 neurons then 256 neurons, each followed by dropout which randomly disables 30% of neurons during training to prevent the model from memorizing rather than learning. The final layer has exactly 27 neurons, one per class, and uses softmax to convert raw scores into probabilities that add up to 1.0. So the output might look like: A = 73%, B = 8%, C = 4%, and so on across all 27 letters. The highest probability is the prediction

The raw prediction changes every single frame which would cause the displayed letter to flicker constantly. To fix this, the last 5 predictions are stored in a buffer and a majority vote is taken. Whichever letter appeared most in those 5 frames is what gets displayed. At 30 frames per second this covers about 167 milliseconds, which is unnoticeable as a delay but gives a stable readable output. The top 3 predictions are shown on screen along with confidence percentages, and if the top prediction falls below 30% confidence it is flagged as low confidence rather than displaying a potentially wrong answer. borrowing the visual intelligence MobileNetV2 already learned and using it to describe our hand images. The output is a compact 1,280-number summary of what the network detected.

Training Data

Why data quality determined real world success

Part 1 — Initial Dataset (The Problem)

- Source: Small ASL image dataset, ~2,500 total images
 - ~70 images per class across 36 classes (digits 0-9 + letters a-z)
- All images had plain black backgrounds — studio style, not real world
- Validation accuracy reached ~75%. Appeared acceptable on paper



Why it failed in practice:

- Small dataset
- Model learned "black background + hand shape = letter" rather than hand shape alone
- This is called domain shift — the training environment and deployment environment look completely different
- When shown a real webcam feed with furniture, walls, and normal lighting, the model had never seen anything like it
- Result: model struggled to accurately predict most signs
- Validation accuracy was misleading. It was measured on other black background images from the same dataset, not real webcam conditions

Training Data

Part 2 — The Fix (Better Data)

- Switched to grassknotted/asl-alphabet dataset on Kaggle
- 87,000 images · 1.1 GB · ~3,000 images per class
- Real world backgrounds (varied lighting, different skin tones, different hand sizes)
- 40x more images per class than the initial dataset

Additional tweaking required:

- Removed nothing class : Contained real room photos with no hand, caused model to predict "nothing" for all webcam input since room backgrounds matched perfectly
- Removed del class : Not a standard ASL fingerspelling sign, not useful for live demo
- Built balanced training subset : Exactly 1,500 images per class sampled randomly before training to prevent any letter from dominating

Why balance mattered:

- Early data pipeline used `.take()` to sample the first half of batches from an alphabetically ordered dataset — earlier letters like A through J were heavily overrepresented while later letters like S through Z were underrepresented or missing entirely. The model learned the early letters well and defaulted to them for everything else. Fix: randomly sample exactly 1,500 images from each class folder individually before training, guaranteeing equal representation across all 27 classes.



Live presentation

Live Presentation of model that was trained on initial dataset with black background versus larger dataset.

-live_asl2.py (old model)

-live_asl3.py (new model)

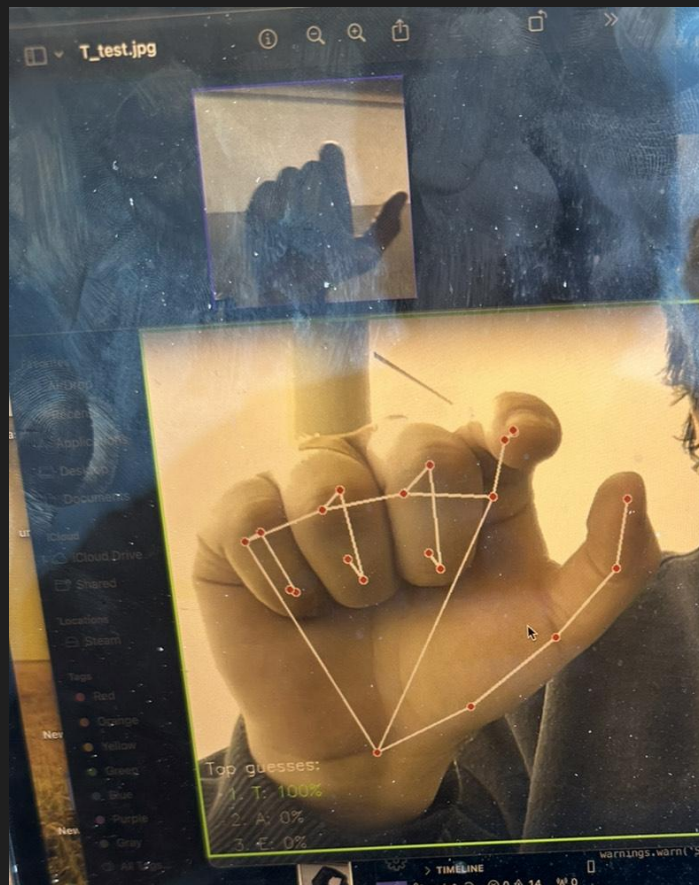


Live presentation

Live Presentation of model that was trained on initial dataset with black background versus larger dataset.

-live_asl2.py (old model)

-live_asl3.py (new model)



End

Questions?